

SAIA 2163 • NATURAL LANGUAGE PROCESSING

Social Media Emotion Analyzer

Detecting six human emotions in short text using classical NLP and machine learning

Final Project Technical Report

Theme 5 — Social Media Emotion Analyzer

Team Members: _____

Contents

1. Introduction

Understanding the emotion behind a message is one of the most practical applications of Natural Language Processing. Brands monitor how customers feel, platforms flag distress, and researchers study public mood at scale. This project builds a complete system that reads a short social-media-style message and predicts the emotion behind it.

The application classifies text into **six emotions**: joy, sadness, love, anger, fear, and surprise. It covers the full lifecycle — from raw labelled data, through preprocessing and modelling, to an interactive web application a non-technical user can operate.

1.1 Objectives

- Collect and prepare a labelled dataset of at least 1,000 text samples.
- Build a reproducible NLP preprocessing pipeline.
- Implement two distinct feature-extraction methods (TF-IDF and Word2Vec).
- Train and compare at least two classification models.
- Deploy the best model in an interactive Streamlit web application.
- Communicate insights through at least five visualizations.

1.2 Dataset

We use the publicly available **Emotion dataset** (originally from HuggingFace), containing 20,000 English messages split into 16,000 training, 2,000 validation, and 2,000 test samples. Each row has a *text* field and an integer *label* mapped to one of the six emotions. Training and validation were combined (18,000 messages) for final model fitting; the 2,000-message test set was held out for evaluation only.

2. Methodology

2.1 Text preprocessing

Every message passes through an identical cleaning pipeline at both training and prediction time, guaranteeing consistent behaviour in the deployed app:

- Lower-casing all text.
- Removing URLs, numbers, and non-alphabetic characters.
- Tokenizing into individual words.
- Removing English stop-words, while **deliberately keeping negations** (“not”, “never”) because they carry emotional meaning.
- Lemmatizing words to their dictionary form (e.g. “feeling” → “feel”).

2.2 Feature extraction

Two complementary representations were implemented so their effect on accuracy could be compared directly:

Method	Description	Library
TF-IDF	Unigrams + bigrams, 8,000 features, sublinear weighting	scikit-learn

Method	Description	Library
Word2Vec	100-dimensional embeddings, averaged per message	gensim

2.3 Models

Three classical classifiers were trained. Class weighting was set to “balanced” to compensate for the under-represented emotions (love and surprise).

- Multinomial Naive Bayes — fast probabilistic baseline (TF-IDF only).
- Logistic Regression — linear, interpretable, outputs probabilities.
- Linear SVM — strong margin-based classifier for sparse text features.

3. Results

All models were evaluated on the untouched 2,000-message test set. The table reports weighted precision, recall, and F1 to account for class imbalance.

Model	Features	Accuracy	Precision	Recall	F1
Naive Bayes	TF-IDF	79.1%	82.5%	79.1%	76.2%
Logistic Regression	TF-IDF	90.0%	90.8%	90.0%	90.2%
Linear SVM	TF-IDF	90.3%	90.7%	90.3%	90.4%
Logistic Regression	Word2Vec	42.6%	51.7%	42.6%	45.1%
Linear SVM	Word2Vec	48.4%	49.4%	48.4%	48.5%



Figure 1 — Accuracy and F1 across all five model/feature combinations.

The strongest result came from **TF-IDF features**. Linear SVM and Logistic Regression both reached roughly 90% accuracy, far ahead of the Naive Bayes baseline (79%). Averaged Word2Vec performed poorly ($\approx 43\text{--}48\%$): because messages are short, averaging word vectors dilutes the strong emotional keywords that TF-IDF preserves directly.

Logistic Regression + TF-IDF was selected for the web application. It matches the top accuracy while also producing calibrated class probabilities, which the app displays as confidence scores.

3.1 Per-class behaviour

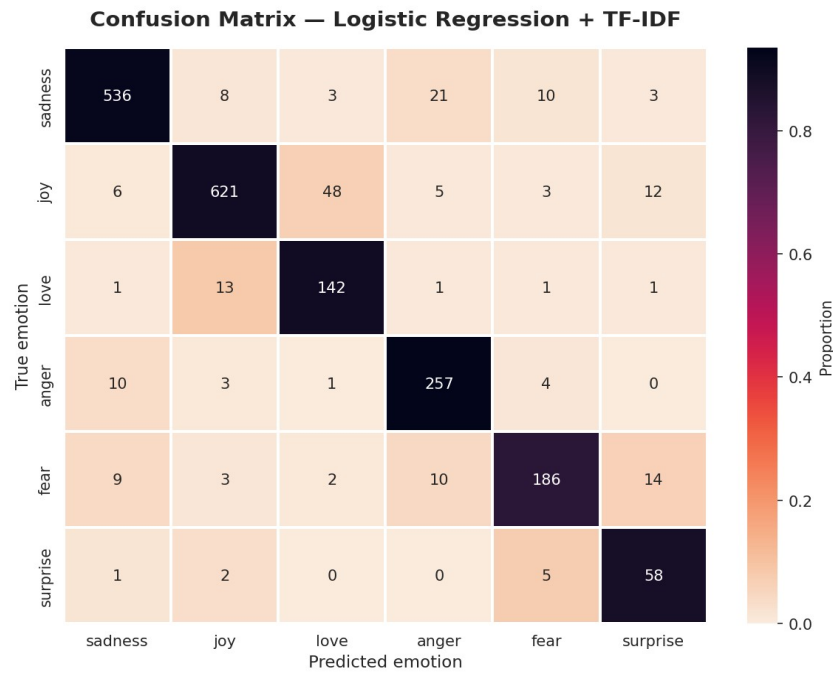


Figure 2 — Confusion matrix for the deployed model (counts; rows = true emotion).

Joy and sadness are recognised almost perfectly. The hardest class is surprise — it is both rare and semantically close to joy and fear, so a portion of surprise messages are absorbed into those classes.

4. Data Insights & Visualizations

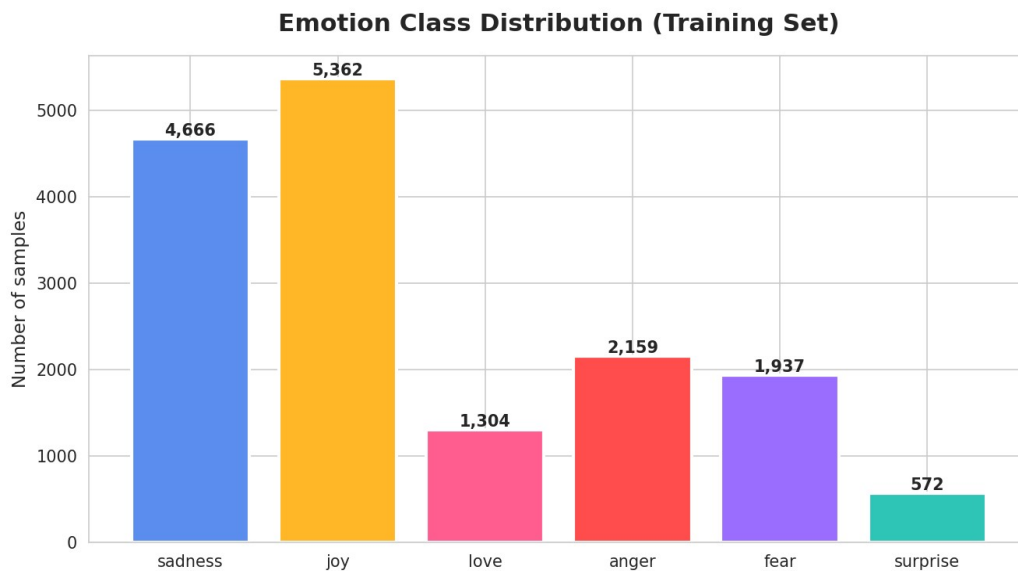


Figure 3 — Emotion class distribution in the training set.

The dataset is imbalanced: joy and sadness together account for over half of all messages, while surprise is the smallest class. This shaped our decision to use balanced class weights.

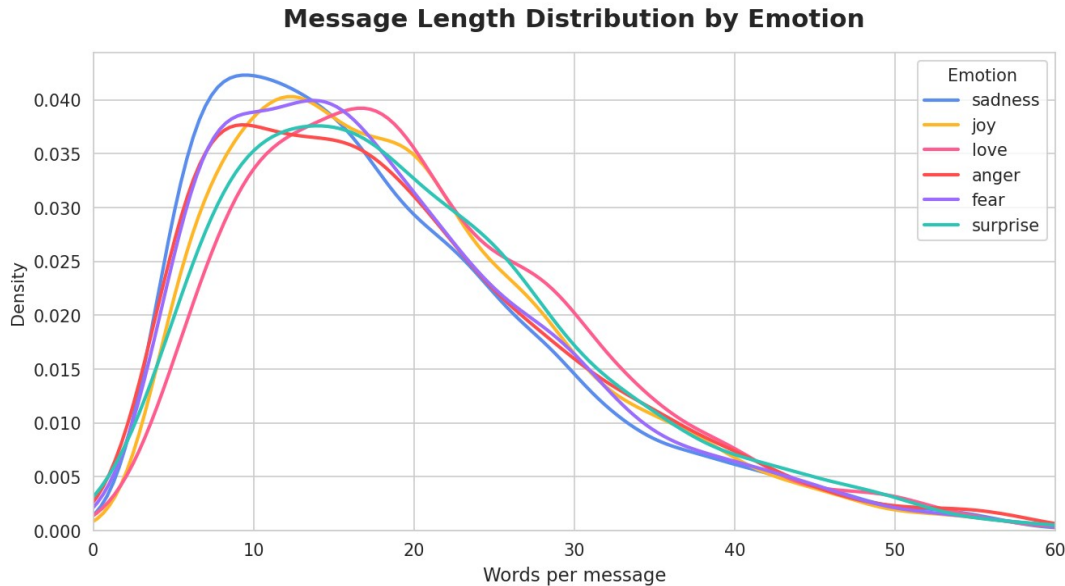


Figure 4 — Message length distribution by emotion.

Most messages are short (median ~17 words), and length is fairly consistent across emotions, confirming that the signal lies in word choice rather than message length.

Most Frequent Words per Emotion



Figure 5 — Most frequent words per emotion after preprocessing.

The word clouds make the learned signal intuitive: “feel” dominates every class, but distinctive markers emerge — “loved/sweet” for love, “scared/afraid” for fear, “angry/hate” for anger.

5. The Web Application

The model is deployed in an interactive Streamlit app with a custom, modern interface. A user types any message and instantly receives:

- The predicted emotion with its emoji and a confidence percentage.
- A live probability bar across all six emotions.
- Interactive dataset insights (distribution, length, word clouds).
- A full performance section (model comparison, confusion matrix).

The app loads the saved *best_model.pkl* and *tfidf_vectorizer.pkl*, applies the identical preprocessing pipeline, and runs entirely locally with *streamlit run app.py*.

6. Challenges & Solutions

Challenge	How we solved it
Class imbalance (surprise rare)	Used <code>class_weight='balanced'</code> in all classifiers.
Negations lost in stop-word removal	Kept negation words so “not happy” stays meaningful.
Word2Vec underperformed on short text	Compared it fairly, reported the finding, used TF-IDF for deployment.
Consistency between training and app	Shared one <code>preprocess.py</code> module across notebook and app.

7. Conclusion

We delivered a complete, working emotion-classification system that reaches 90% test accuracy and is usable by anyone through a polished web interface. The project demonstrates the full NLP workflow: preprocessing, two feature-extraction methods, multiple compared models, rigorous evaluation, and deployment.

7.1 Limitations

- English-only; performance on other languages is untested.
- Surprise remains hard to detect due to scarcity and overlap.
- Classical features miss deeper context a transformer could capture.

7.2 Future work

- Fine-tune a transformer (e.g. BERT) for higher accuracy.
- Apply oversampling (SMOTE) to strengthen minority classes.
- Add multi-language support and emoji-aware features.

8. References

- Saravia et al. (2018). CARER: Contextualized Affect Representations for Emotion Recognition. EMNLP.
- Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. JMLR.
- Řehůřek & Sojka (2010). Gensim: Topic Modelling for Humans.
- Streamlit Inc. (2024). Streamlit Documentation. <https://docs.streamlit.io>